



HYBRID AGENTIC SOC

Rule Engine + AI Agent Investigation

Proof of Concept — Technical Documentation

Field	Details
Author	Muhammad Ali
Role	Junior SOC Engineer — Ebryx Pvt. Ltd., Lahore, Pakistan
Date	August 2026
Version	v1.0
Classification	Internal / Portfolio
GitHub	github.com/Security-Eng-Muhammad-Ali/hybrid-agentic-soc

Phase 1: Executive Summary

1.1 Problem Statement

Traditional SOC analysts face a critical challenge known as "alert fatigue." Modern Security Operations Centers generate hundreds to thousands of security alerts daily, of which 70-90% are low-risk, repetitive events requiring no human intervention. This creates three major problems:

- Analyst Burnout — Reviewing every alert manually leads to fatigue and reduced effectiveness
- Missed Threats — Critical high-risk alerts get buried under low-risk noise
- Slow Response — Manual investigation of each alert takes 30-40 minutes per case

1.2 Solution Overview

This project implements a Hybrid Agentic SOC — a pipeline that combines deterministic rule-based routing with AI-powered threat investigation. The core principle:

- Rules handle routine, repetitive alerts automatically (70-90% of volume)
- AI investigates only high-risk alerts that need deep analysis (5-10% of volume)
- Humans make final decisions on confirmed threats — with one command to block

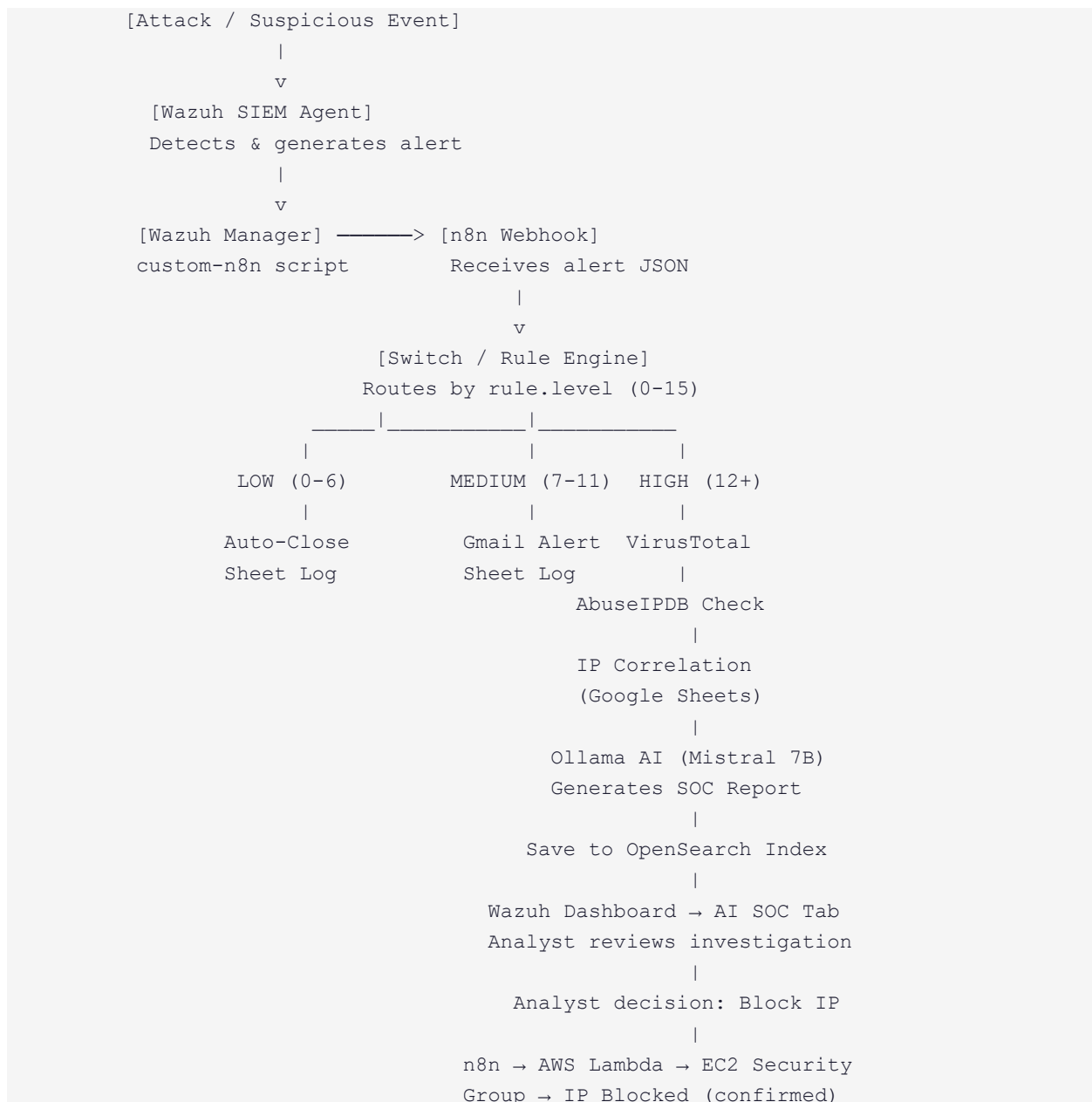
1.3 Key Achievements

Metric	Before (Manual)	After (Automated)
Alert Review Time	30-40 min/alert	Automated in seconds
Analyst Alert Load	100% of all alerts	Only HIGH risk (5-10%)
Threat Intelligence	Manual lookup required	Auto-enriched (VT + AbuseIPDB)
IP Blocking	Manual SG rule creation	Automated via AWS Lambda
AI Cost	Per-call external API pricing	Local model — zero marginal cost
Data Privacy	External API data exposure	100% on-premise (AWS VPC)
Audit Trail	Manual or none	Automated Google Sheets logging

Phase 2: Architecture & Design

2.1 High-Level Architecture

The system follows a multi-tier architecture deployed entirely on AWS EC2 instances within the same VPC for secure internal communication:



2.2 AWS Infrastructure

Instance Name	EC2 Type	Purpose	Region
Wazuh Server	t3.medium	SIEM Manager + OpenSearch + Dashboard	eu-north-1

Linux Agent	t2.micro	Target server with Wazuh Agent installed	eu-north-1
n8n Server	t2.micro	Automation engine + Rule Engine (Docker)	eu-north-1
Ollama Server	t3.large	Local AI Model host — Mistral 7B (8GB RAM)	eu-north-1

2.3 Complete Tech Stack

Component	Tool / Service	Purpose
SIEM	Wazuh v4.14	Alert detection, log collection, FIM monitoring
Automation	n8n (self-hosted, Docker)	Rule engine, workflow orchestration, API calls
AI Model	Ollama + Mistral 7B	Local LLM — zero external API cost, full privacy
Threat Intel 1	VirusTotal API (free tier)	IP malicious engine count + reputation score
Threat Intel 2	AbuseIPDB API (free tier)	IP abuse score, country, ISP, Tor node status
Storage	OpenSearch (Wazuh built-in)	wazuh-ai-summaries custom index
Dashboard	Wazuh OpenSearch Dashboards	AI SOC Investigations custom tab
Notifications	Gmail SMTP via n8n	Medium risk analyst email alerts
Correlation	Google Sheets (6 tabs)	Alert log, IP/Agent/Pattern/Chain tracking
IP Blocking	AWS Lambda + EC2 API	Automated security group rule creation
Infrastructure	AWS EC2 (eu-north-1)	All components hosted, same VPC

Phase 3: Implementation — Step by Step

Phase 3.1 — Wazuh SIEM Setup

Step 1: Wazuh Manager Installation

Deploy Wazuh all-in-one (Manager + OpenSearch + Dashboard) on t3.medium EC2:

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
sudo bash wazuh-install.sh -a
```

Step 2: Wazuh Agent Deployment

Install Wazuh agent on the target Linux server:

```
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.5-1_amd64.deb
sudo WAZUH_MANAGER="<MANAGER_PRIVATE_IP>" dpkg -i wazuh-agent_4.14.5-1_amd64.deb
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
```

Step 3: File Integrity Monitoring (FIM) Configuration

Add monitored directories to `/var/ossec/etc/ossec.conf` on the agent:

```
<syscheck>
  <directories check_all="yes" realtime="yes">/etc</directories>
  <directories check_all="yes" realtime="yes">/tmp</directories>
  <directories check_all="yes" realtime="yes">/var/log</directories>
  <directories check_all="yes" realtime="yes">/usr/bin</directories>
</syscheck>
```

Step 4: Custom High-Risk Detection Rules

Create `/var/ossec/etc/rules/local_rules.xml` on the Wazuh manager:

```
<group name="local,syslog,sshd,">

  <!-- Triggers on 8+ SSH failures from same IP within 60 seconds -->
  <!-- MITRE ATT&CK: T1110 - Brute Force -->
  <rule id="100001" level="12" frequency="8" timeframe="60">
    <if_matched_sid>5710</if_matched_sid>
    <same_source_ip />
    <description>High Risk: Multiple SSH brute force attempts</description>
    <mitre><id>T1110</id></mitre>
  </rule>

  <!-- Triggers on any new user account creation -->
  <!-- MITRE ATT&CK: T1136 - Create Account -->
  <rule id="100002" level="13">
    <if_sid>5901</if_sid>
    <description>High Risk: New user added to system</description>
    <mitre><id>T1136</id></mitre>
```

```
</rule>

</group>
```

Restart Wazuh manager to apply rules:

```
sudo systemctl restart wazuh-manager
```

Phase 3.2 — n8n Automation Setup

Step 1: n8n Installation via Docker

```
docker run -d \
  --name n8n \
  -p 5678:5678 \
  -v n8n_data:/home/node/.n8n \
  -e N8N_SECURE_COOKIE=false \
  -e WEBHOOK_URL=http://<PUBLIC_IP>.nip.io:5678/ \
  --restart unless-stopped \
  n8nio/n8n
```

Step 2: Create Wazuh → n8n Integration Script

Create `/var/ossec/integrations/custom-n8n` on the Wazuh manager:

```
#!/bin/sh
WEBHOOK_URL="http://<N8N_PRIVATE_IP>:5678/webhook/wazuh-alerts"
ALERT_FILE=$1
curl -s -X POST -H "Content-Type: application/json" -d @${ALERT_FILE} ${WEBHOOK_URL}
exit 0
```

Set correct permissions:

```
sudo chmod +x /var/ossec/integrations/custom-n8n
sudo chown root:wazuh /var/ossec/integrations/custom-n8n
```

Step 3: Add Integration to ossec.conf

Add the following block inside `<ossec_config>` in `/var/ossec/etc/ossec.conf`:

```
<integration>
  <name>custom-n8n</name>
  <hook_url>http://<N8N_PRIVATE_IP>:5678/webhook/wazuh-alerts</hook_url>
  <level>3</level>
  <alert_format>json</alert_format>
</integration>
```

Phase 3.3 — n8n Workflow: Alert Router

Workflow Name: "Hybrid Agentic SOC"

This is the main workflow. Build the following node chain in n8n canvas:

Node Name	Node Type	Configuration
Webhook	Webhook Trigger	Method: POST Path: wazuh-alerts
Switch	Switch (Rules mode)	LOW: level <= 6 MEDIUM: level 7-11 HIGH: level >= 12
VirusTotal IP Check	HTTP Request	GET https://www.virustotal.com/api/v3/ip_addresses/{srcip} Header: x-apikey
AbuseIPDB Check	HTTP Request	GET https://api.abuseipdb.com/api/v2/check?ipAddress={srcip}&maxAgeInDays=90 Header: Key
Get IP History	Google Sheets	Operation: Get Rows Sheet: IP_Correlation Filter by Source IP
Ollama AI	HTTP Request	POST http://<OLLAMA_PRIVATE_IP>:11434/api/generate Body: enriched prompt JSON
Save to OpenSearch	HTTP Request	POST https://<WAZUH_IP>:9200/wazuh-ai-summaries/_doc Auth: Basic
Send a message	Gmail	To: analyst email Subject: Medium Risk Alert Body: alert details
Append row in sheet	Google Sheets	Operation: Append Row Sheet: SOC Alert Log (Sheet1)

Switch Node Routing Rules (in order):

Priority	Condition	Output Branch	Action
1st	rule.level >= 12	HIGH	VirusTotal → AbuseIPDB → IP History → AI → OpenSearch
2nd	rule.level >= 7	MEDIUM	Gmail notification + Google Sheet log
3rd	rule.level <= 6	LOW	Google Sheet auto-log only

Ollama AI Prompt Template:

The following JSON is sent to Mistral 7B in the Ollama AI node body:

```
{
  "model": "mistral",
  "stream": false,
  "prompt": "You are a SOC analyst. Analyze this security alert with
            threat intelligence and historical correlation data.

            ALERT DETAILS:
            - Severity Level: {{ rule.level }}
            - Description: {{ rule.description }}
            - Agent: {{ agent.name }} ({{ agent.ip }})
            - Source IP: {{ data.srcip }}
            - Timestamp: {{ timestamp }}

            THREAT INTELLIGENCE:
            VirusTotal: {{ malicious }}/{{ total }} engines flagged
            Reputation Score: {{ reputation }}
            AbuseIPDB Score: {{ abuseConfidenceScore }}%
            Total Reports: {{ totalReports }}
            Country: {{ countryCode }} | ISP: {{ isp }}
            Tor Exit Node: {{ isTor }}"
```

HISTORICAL CORRELATION:

Previous alerts from this IP: {{ Total Alerts }}

First seen: {{ First Seen }}

Previous HIGH alerts: {{ High Count }}

Known attack types: {{ Attack Types }}

Provide:

1) THREAT SUMMARY (with actual TI scores)

2) RISK ASSESSMENT

3) RECOMMENDED ACTIONS (specific, actionable)"

}

Phase 3.4 — Ollama AI Setup

Step 1: Install Ollama on t3.large EC2

```
curl -fsSL https://ollama.com/install.sh | sh
ollama pull mistral
```

Step 2: Configure Ollama to Listen on All Interfaces

By default, Ollama only listens on localhost. Edit the systemd service:

```
sudo systemctl edit ollama.service
```

Add the following in the editor that opens:

```
[Service]
Environment="OLLAMA_HOST=0.0.0.0"
```

Apply the changes:

```
sudo systemctl daemon-reload
sudo systemctl restart ollama
```

Step 3: Verify Ollama is Listening on All Interfaces

```
sudo ss -tlnp | grep 11434
```

Expected output: *:11434 (confirms listening on all interfaces, not just localhost)

Step 4: Test from n8n Server

```
curl http://<OLLAMA_PRIVATE_IP>:11434/api/generate \
-d '{"model": "mistral", "prompt": "Say hello", "stream": false}'
```

Phase 3.5 — OpenSearch Custom Index

Step 1: Create wazuh-ai-summaries Index

Run on the Wazuh Manager server:

```
curl -k -u admin:'<WAZUH_INDEXER_PASSWORD>' \
-X PUT "https://localhost:9200/wazuh-ai-summaries" \
-H "Content-Type: application/json" \
-d '{
  "mappings": {
    "properties": {
      "timestamp": {"type": "date"},
      "source_ip": {"type": "keyword"},
      "agent_name": {"type": "keyword"},
      "rule_level": {"type": "integer"},
      "rule_description": {"type": "text"},
      "virustotal_score": {"type": "integer"},
      "abuseipdb_score": {"type": "integer"},
      "abuseipdb_country": {"type": "keyword"},
    }
  }
}
```

```
"is_tor":          {"type": "boolean"},
"ai_summary":      {"type": "text"},
"status":          {"type": "keyword"}
}
}
}'
```

Step 2: Find Wazuh Indexer Password

```
sudo cat /root/wazuh-passwords.txt 2>/dev/null
```

Phase 3.6 — Wazuh Dashboard: AI SOC Investigations Tab

Step 1: Create Index Pattern

In Wazuh Dashboard: Stack Management → Index Patterns → Create index pattern

- Index pattern name: wazuh-ai-summaries
- Time field: timestamp
- Click "Create index pattern"

Step 2: Create Saved Search (Main Table)

In Discover: select wazuh-ai-summaries index, add these columns in order:

source_ip → rule_level → agent_name → virustotal_score → abuseipdb_score → abuseipdb_country → is_tor → rule_description → ai_summary → status

Save as: "AI SOC Investigations Table"

Step 3: Create Dashboard Panels

Panel Name	Visualization Type	Configuration
AI SOC Investigations Table	Saved Search	Use saved search from Step 2
Pending Reviews	Metric (Count)	Filter: status = "Pending Review"
Total AI Investigations	Metric (Count)	No filter — counts all documents
Alert Status Breakdown	Pie Chart	Terms aggregation on "status" field

Step 4: Expand Rows to Read Full AI Summary

In the table, click the ">" arrow on any row to expand and read the complete AI investigation report. The ai_summary field contains the full 3-part analysis (Threat Summary, Risk Assessment, Recommended Actions).

Phase 3.7 — Automated IP Blocking via AWS Lambda

Step 1: Create IAM User for n8n

- IAM → Users → Create user: n8n-soc-automation
- Attach policy: AmazonEC2FullAccess
- Create Access Key → copy Access Key ID and Secret Access Key

Step 2: Create Lambda Function

- Runtime: Python 3.12 | Name: soc-ip-block
- Enable Function URL (Auth: NONE)
- Set Timeout to 30 seconds
- Attach IAM role with AmazonEC2FullAccess

Step 3: Lambda Function Code

```
import boto3, json

def lambda_handler(event, context):
    body = json.loads(event.get("body", "{}"))
    ip      = body.get("ip_address")
    action  = body.get("action")          # "block" or "unblock"
    sg_id   = body.get("security_group_id")
    reason  = body.get("reason", "SOC AI Investigation")

    ec2 = boto3.client("ec2", region_name="eu-north-1")

    if action == "block":
        ec2.authorize_security_group_ingress(
            GroupId=sg_id,
            IpPermissions=[{
                "IpProtocol": "tcp",
                "FromPort": 0, "ToPort": 65535,
                "IpRanges": [{
                    "CidrIp": f"{ip}/32",
                    "Description": f"BLOCKED: {reason}"
                }]
            }]
        )
    return {
        "statusCode": 200,
        "body": json.dumps({"message": f"IP {ip} {action}ed successfully"})
    }
```

Step 4: Create n8n Workflow "SOC IP Block Action"

Node	Type	Configuration
------	------	---------------

Webhook	Trigger	Path: /soc-action Method: POST Receives: {ip_address, action, security_group_id, reason}
Call Lambda	HTTP Request	POST to Lambda Function URL Body: {ip_address, action, security_group_id, reason}
Update OpenSearch	HTTP Request	POST to /wazuh-ai-summaries/_update_by_query Updates status field
Log Action	Google Sheets	Append to Action_Log tab with timestamp, IP, action, reason, status

Step 5: Block an IP (Analyst Command)

When analyst confirms True Positive, run from n8n server:

```
curl -X POST http://localhost:5678/webhook/soc-action \
-H "Content-Type: application/json" \
-d '{
  "ip_address": "<ATTACKER_IP>",
  "action": "block",
  "security_group_id": "<SG_ID>",
  "reason": "SOC AI Investigation - True Positive - SSH Brute Force"
}'
```

Phase 3.8: Native OpenSearch Dashboards Plugin

3.8.1 Overview

While Phase 3.6 delivers the AI SOC Investigations data via a standard Discover saved search and Dashboard, this phase upgrades that experience into a fully native OpenSearch Dashboards plugin — a custom sidebar application, not a saved dashboard link. This gives the AI SOC Investigations view its own dedicated navigation entry, custom branding, and a live status header, matching the integration depth of Wazuh's own built-in modules.

3.8.2 Why a Native Plugin

A saved dashboard is accessible but generic — it looks like any other dashboard in the platform. A native plugin:

- Appears as a first-class item in the Wazuh sidebar navigation, under its own category
- Carries a custom header showing live pipeline context (AI Model, Threat Intel sources, Auto-Action capability) at a glance
- Is versioned and deployable independently of the OpenSearch Dashboards build process — no rebuild of the entire platform required

3.8.3 Implementation Approach

OpenSearch Dashboards (a fork of Kibana) loads legacy-style plugins via a webpack bundle convention: each plugin ships a compiled `<plugin-id>.plugin.js` file that registers itself with the platform's module loader using `__osdBundles__.define(bundleId, requireFn, entryModuleId)`. This is undocumented at the integration level — it had to be reverse-engineered by inspecting Wazuh's own compiled core plugin (`wazuhCore.plugin.js`) to identify the correct registration pattern.

Plugin structure:

```
opensearch-dashboards/wazuhAiSoc/  
├─ package.json  
├─ opensearch_dashboards.json (plugin manifest: id, version, ui: true)  
├─ server/  
│   └─ index.js  
│   └─ plugin.js  
└─ target/public/  
    └─ wazuhAiSoc.plugin.js (compiled UI bundle)
```

The public bundle registers a new application via `core.application.register()`, embedding the existing Phase 3.6 dashboard (by saved-object ID) inside an iframe with a custom header and live

status bar, rather than duplicating the underlying visualization logic.

3.8.4 Deployment Process

```
curl -sO https://packages.wazuh.com/... # N/A — plugin is custom-built,  
# not downloaded
```

Package and deploy:

```
zip -r wazuhAiSoc.zip opensearch-dashboards/  
sudo cp -r opensearch-dashboards/wazuhAiSoc /usr/share/wazuh-dashboard/plugins/  
sudo chown -R wazuh-dashboard:wazuh-dashboard /usr/share/wazuh-dashboard/plugins/wazuhAiSoc  
sudo systemctl restart wazuh-dashboard
```

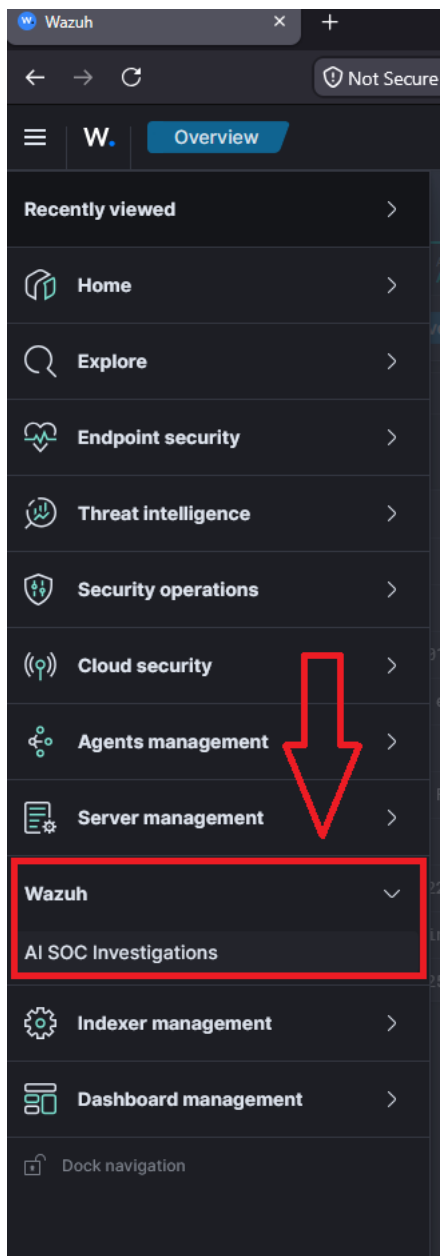
3.8.5 Engineering Challenges Resolved

- Bundle registration failure ("Definition of plugin wazuhAiSoc not found and may have failed to load"): root-caused to a missing `__osdBundles__.define()` call — the initial implementation used a non-standard window-global convention that the platform's core loader does not recognize.
- Iframe row-rendering failure: the embedded dashboard's virtualized data grid measured a zero-height container on mount inside the flex-based iframe wrapper, silently rendering zero rows despite the underlying saved search returning results correctly. Resolved by giving the iframe an explicit min-height.
- Terminal paste corruption during file transfer over browser-based EC2 Instance Connect: large multi-line pastes were interleaved/truncated by the terminal. Resolved by transferring the plugin bundle as a single base64-encoded line and decoding server-side.

3.8.6 Result

The AI SOC Investigations view is now accessible as a dedicated Wazuh sidebar tab (Wazuh → AI SOC Investigations), independent of the generic Dashboards section, displaying the same wazuh-ai-summaries index data established in Phase 3.5–3.6.





Wazuh

Overview

AI SOC Investigations LIVE

AI MODEL
Mistral 7B (Local)

THREAT INTEL
VirusTotal + AbuseIPDB

AUTO-ACTION
AWS Lambda IP Block

W.

Dashboards

AI SOC Investigations

Full screen

Share

Clone

Reporting

Edit

a

⌵

Search

DQL

Last 7 days

Show dates

Refresh

⌵

Add filter

AI SOC Investigations Table

abuseipdb_country	DE
abuseipdb_score	100
agent_name	node01
ai_summary	Test entry - SSH brute force from confirmed Tor exit node. Sample data for dashboard verification.
is_tor	true
rule_description	High Risk: Multiple SSH brute force attempts
rule_level	12
source_ip	185.228.101.45
status	Pending Review
timestamp	Aug 25, 2026 @ 11:17:59.000

11:54 AM

8/25/2026

Phase 4: Correlation Engine

4.1 Overview

The correlation engine automatically tracks attack patterns across multiple dimensions in Google Sheets. This historical data is retrieved by n8n before calling the AI, giving Mistral 7B full context about whether an attacker is a repeat offender.

4.2 Google Sheets Structure (SOC Alert Log)

Tab Name	What It Tracks	Key Columns
Sheet1 (SOC Alert Log)	Every alert from Wazuh	Timestamp, Rule Level, Description, Agent Name, Agent IP, Source IP, Status (AI summary for HIGH)
IP_Correlation	Per source IP history	Source IP, First Seen, Last Seen, Total Alerts, Low Count, Medium Count, High Count, Attack Types, Status
Agent_Correlation	Which agents are targeted	Agent Name, Agent IP, Total Alerts, Low/Med/High Count, Most Common Attack
Pattern_Correlation	Which attacks repeat most	Attack Pattern, Rule ID, First/Last Seen, Total Count, Affected Agents, Affected IPs
Attack_Chain	Multi-stage attack progression	Source IP, Chain Start/End, Low/Med/High Alerts, Total Alerts, Chain Status
Action_Log	All analyst decisions	Timestamp, IP Address, Action, Reason, Status, Blocked By

4.3 How Correlation Improves AI Analysis

Before calling Ollama AI, the "Get IP History" node queries the IP_Correlation tab for the source IP. The result is injected into the AI prompt:

- If the IP has 0 previous alerts: AI treats it as a first-time attacker
- If the IP has 5+ HIGH alerts: AI escalates its threat assessment to critical
- If the IP has a history of Low → Medium → High progression: AI identifies it as a multi-stage attack chain

Phase 5: Real-World Results

5.1 Real Attackers Detected During Testing

The system detected real external attackers attempting to compromise the AWS EC2 instances during the deployment period:

Source IP	VT Score	AbuseIPDB	Country	Attack Type	Action
185.220.101.45	17/91 engines	100% confidence	Germany (DE)	SSH Brute Force — Tor Exit Node	IP Blocked via Lambda
161.118.212.147	Multiple detections	High score	External	SSH Brute Force (20+ attempts)	Logged
87.152.6.217	Detected	High score	External	SSH Scan Attempt	Logged
198.235.24.108	Detected	Medium score	External	Connection Reset Attack	Logged
190.161.133.46	Detected	High score	External	SSH Connection Reset	Logged

5.2 Sample AI Investigation Report

Actual output from Mistral 7B for IP 185.220.101.45 (Tor Exit Node):

```
TITLE: Security Alert Report — SSH Brute Force from Confirmed Malicious IP

1) THREAT SUMMARY:
Multiple SSH brute force attempts detected from 185.220.101.45 targeting
Linux-agent (172.31.0.189). VirusTotal flagged by 17/91 engines as malicious
(reputation score: -20). AbuseIPDB: 100% abuse confidence, 143 abuse reports,
Germany (DE). IP is a confirmed Tor Exit Node (ISP: for-privacy.net).
REPEAT ATTACKER — previously seen on Aug 7, 2026 with multiple HIGH alerts.

2) RISK ASSESSMENT:
CRITICAL — Active brute force from confirmed Tor exit node. Maximum abuse
confidence score combined with repeat attack history indicates a persistent,
sophisticated threat actor using Tor for obfuscation. High likelihood of
credential stuffing or targeted intrusion attempt.

3) RECOMMENDED ACTIONS:
- Block 185.220.101.45 at AWS Security Group level immediately
- Review /var/log/auth.log for any successful SSH logins from this IP
- Implement SSH key-based authentication only (disable password auth)
- Enable SSH rate limiting (fail2ban or AWS WAF)
- Add this IP to threat intel blocklist
- Consider implementing Tor exit node blocklist service
```

Phase 6: MITRE ATT&CK Coverage

The following MITRE ATT&CK techniques are detected and mapped by this system:

Technique ID	Technique Name	Detection Method	Wazuh Rule
T1110	Brute Force	SSH failed login frequency correlation	Custom Rule 100001 (level 12)
T1110.001	Password Guessing	Invalid user SSH login attempts	Wazuh Rule 5710 (level 5)
T1021.004	SSH Remote Services	SSH connection monitoring	Wazuh Rule 5710 family
T1136	Create Account	New Linux user account detection	Custom Rule 100002 (level 13)
T1078	Valid Accounts	Successful root login detection	Wazuh Rule 5402
T1222	File & Directory Permissions Modification	FIM via Wazuh Syscheck on /etc	Wazuh Rule 550 (level 7)
T1565	Data Manipulation	Integrity checksum change detection	Wazuh Rule 550 (level 7)

Phase 7: Future Improvements

Feature	Description	Priority
Ticketing Integration	Auto-create Jira/ServiceNow tickets for HIGH alerts with AI summary pre-filled	High
Slack Notifications	Send HIGH alert cards to SOC team Slack channel with approve/block buttons	High
Multi-Agent Correlation	Track same attacker IP across all EC2 instances in the environment	High
Auto-Unblock Schedule	Automatically review and remove IP blocks after 30 days with re-evaluation	Medium
False Positive Tuning	Analyst rejection feedback loop to improve AI prompt accuracy over time	Medium
Multi-Model Support	Support GPT-4/Claude API for premium investigation when Mistral is uncertain	Medium
Threat Hunting Queries	AI auto-generates Wazuh hunting queries based on investigation findings	Low
SOAR Integration	Full integration with enterprise SOAR platforms (Splunk SOAR, Palo Alto XSOAR)	Low

Phase 8: Appendix

8.1 Required API Keys & Credentials

Service	URL	Plan Used	Rate Limits
VirusTotal	virustotal.com	Free	4 requests/min, 500 requests/day
AbuseIPDB	abuseipdb.com	Free	1,000 requests/day
Google Cloud	console.cloud.google.com	Free	Sheets API + Gmail API

8.2 AWS Resources Summary

Resource	Identifier / Config	Purpose
EC2 Instances (4)	eu-north-1, same VPC	Wazuh, Agent, n8n, Ollama
Security Group	sg-03d1ab1e31e88b726	Agent server — IP blocking target
IAM User	n8n-soc-automation	Lambda EC2 API access (AmazonEC2FullAccess)
Lambda Function	soc-ip-block (Python 3.12)	30s timeout, Function URL enabled
OpenSearch Index	wazuh-ai-summaries	Custom AI investigation summaries storage

8.3 Attack Simulation Script

A Python script is included in the GitHub repository to simulate LOW, MEDIUM, and HIGH risk attacks for testing the full pipeline:

```
# Clone the repository
git clone https://github.com/Security-Eng-Muhammad-Ali/hybrid-agentic-soc

# Run on the Wazuh agent server
sudo python3 attack_simulation.py
```

The script simulates: failed SSH logins (LOW), FIM violations (MEDIUM), SSH brute force + user creation (HIGH).

8.4 GitHub Repository

Full source code, configuration files, and documentation:

```
https://github.com/Security-Eng-Muhammad-Ali/hybrid-agentic-soc
```

Repository contains: README.md, attack_simulation.py, custom-n8n script, local_rules.xml, and this POC document.

8.5 Author Information

Field	Details
Name	Muhammad Ali
Role	Junior SOC Engineer / SOC Intern

Organization	Ebryx Pvt. Ltd., Arfa Software Technology Park, Lahore, Pakistan
Education	BS Software Engineering — UBIT, University of Karachi
GitHub	github.com/Security-Eng-Muhammad-Ali
Specialization	SOC Operations, Detection Engineering, SIEM, Cloud Security, AI-Assisted Security